

# CSC 112: Computer Operating Systems

## Review Questions

Department of Computer Science,  
Hofstra University

## Q. Fork

---

- For these questions, assume there is no error, i.e., all fork calls succeed, and the return value of fork() is never negative.
- 1. Will the parent and child print the same value for a?
- ANS: Yes. Processes do not share the same memory space, so a is 2 for both.
- 2. Will they print the same memory address &a?
- ANS: Yes. Fork copies the address space of the parent to the child.
- 3. Will they write to the same STDOUT?
- ANS: Yes. File descriptors are copied over to the new process, so both STDOUTs will reference the same “file”.

```
1 int main(void) {
2     int a = 1;
3     pid_t fork_ret = fork();
4     if (fork_ret > 0) {
5         a++;
6         fprintf(stdout, "Parent: int a is %d at %p\n",
7             a, &a);
8     } else if (fork_ret == 0) {
9         a++;
10        fprintf(stdout, "Child: int a is %d at %p\n", a,
11            &a);
12    } else {
13        printf("Oedipus");
14    }
```

## Q. Fork

- For these questions, assume there is no error, i.e., all fork calls succeed, and the return value of fork() is never negative.
- 1. What does this program print?
- ANS: Currently, the program stops after printing 3, giving an output of
  - 0
  - 1
  - 2
  - 3
  - <output of ls>

```
1 int main(void) {
2 char** argv = (char**) malloc(3 * sizeof(char*));
3 argv[0] = "/bin/ls";
4 argv[1] = ".";
5 argv[2] = NULL;
6 for (int i = 0; i < 10; i++) {
7     printf("%d\n", i);
8     if (i == 3) {
9         execv("/bin/ls", argv);
10    }
11 }
12 return 0;
13 }
```

## Q. Fork

- For these questions, assume there is no error, i.e., all fork calls succeed, and the return value of fork() is never negative.
- 1. What does this program print?
- ANS: The parent process will print from 0 to 9. The child process will print <output of ls> with any possible interleaving with the parent's printing of 0 to 9.
  - 0
  - 1
  - 2
  - ...
  - 9
  - <output of ls>

```
1 int main(void) {
2 char** argv = (char**) malloc(3 * sizeof(char*));
3 argv[0] = "/bin/ls";
4 argv[1] = ".";
5 argv[2] = NULL;
6 for (int i = 0; i < 10; i++) {
7     printf("%d\n", i);
8     if (i == 3) {
9         pid_t fork_ret = fork();
10        if (fork_ret == 0)
11            execv("/bin/ls", argv);
12    }
13 }
14 return 0;
15 }
```

## Q. Dining Lawyers

---

- Consider the Dining Lawyers problem. There are 3 lawyers P1 to P3, each with a different number of arms. P1 has 1 arm and needs 1 fork to eat; P2 has 2 arms and needs 2 forks to eat; P3 has 3 arms and needs 3 forks to eat. There is a pile of 3 forks at center of the table. Each lawyer picks up one fork at a time, and when he gets enough forks, he eats and then puts down all his forks.
- Is it possible for the system to be deadlocked? If no, explain why. If yes, show a (potential) deadlock state and run Banker's algorithm to check it. (You need to give the Max, Allocation, Need matrices, Total and Available vectors, and Available resources after completion of each process.)

## Q. Dining Lawyers Solution: 3 Lawyers, each with 1,2,3 arms, 3 forks

Initially, all forks are free.

Max Allocation

|   |   |
|---|---|
| 1 | 0 |
| 2 | 0 |
| 3 | 0 |

P2 grabs 1 fork  
and P3 grabs 2  
forks

Max Allocation Need

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 2 | 1 | 1 |
| 3 | 2 | 1 |

Total Available

|   |   |
|---|---|
| 3 | 3 |
|---|---|

Total Available

|   |   |
|---|---|
| 3 | 0 |
|---|---|

Available resources after  
completion of each process

|          |    |
|----------|----|
|          | R1 |
| Init     | 0  |
| Deadlock |    |

Current state is a  
deadlock.

## Q. Paging

---

- Consider 12-bit virtual address space. Consider a machine with a page size of 1 KB. There are 8 KB of physical memory and 8KB of virtual memory. The TLB is a fully associative cache with space for 4 entries that is currently empty.
- Assume that the physical page number is always one more than the virtual page number. This is a sequence of memory address accesses for a program:
  - 0x294, 0xA76, 0x5A4, 0x923, 0xCFF, 0xA12, 0xF9F, 0x392, 0x341.
- Here is the current state of the page table.
- How many TLB hits and page faults are there? What are the contents of the TLB at the end of the sequence?

| VPN | Valid | PPN |
|-----|-------|-----|
| 0   | 0     | /   |
| 1   | 1     | 2   |
| 2   | 0     | /   |
| 3   | 0     | 4   |
| 4   | 0     | 5   |
| 5   | 1     | 6   |
| 6   | 1     | 7   |
| 7   | 0     | /   |

Page Table

## Q. Paging ANS

- Given:

- Page size: 1024 bytes (1KB)
- Physical memory: 8KB (8 frames)
- Virtual memory: 8KB (8 pages)
- TLB: fully associative, 4 entries, initially empty
- Physical page number = virtual page number + 1
- Page size is 1 KB =  $2^{10}$ , hence offset is 10 bits. 12-bit virtual address space, hence bits used for VPN indexing is  $12-10=2$  bits. VPN is the left-most 2 bits of the virtual address.
- Looking up the page table, VPNs 1, 5, 6 are valid; others are invalid or marked as NULL.

| Virtual Addr | VPN | TLB  | Memory |
|--------------|-----|------|--------|
| 0x294        | 0   | miss | PF     |
| 0xA76        | 2   | miss | PF     |
| 0x5A4        | 1   | miss | valid  |
| 0x923        | 2   | hit  | valid  |
| 0xCFF        | 3   | miss | PF     |
| 0xA12        | 2   | hit  | valid  |
| 0xF9F        | 3   | hit  | valid  |
| 0x392        | 0   | hit  | valid  |
| 0x341        | 0   | hit  | valid  |

| VPN | Valid | PPN |
|-----|-------|-----|
| 0   | 0     | /   |
| 1   | 1     | 2   |
| 2   | 0     | /   |
| 3   | 0     | 4   |
| 4   | 0     | 5   |
| 5   | 1     | 6   |
| 6   | 1     | 7   |
| 7   | 0     | /   |

Page Table

## Q. Paging ANS: TLB and Page Table at the End

---

| VPN | Valid | PPN | Page<br>Table |
|-----|-------|-----|---------------|
| 0   | 1     | 1   |               |
| 1   | 1     | 2   |               |
| 2   | 1     | 3   |               |
| 3   | 1     | 4   |               |
| 4   | 0     | 5   |               |
| 5   | 1     | 6   |               |
| 6   | 1     | 7   |               |
| 7   | 0     | /   |               |

| Tag | PPN |
|-----|-----|
| 0   | 1   |
| 2   | 3   |
| 1   | 2   |
| 3   | 4   |

TLB

## Q. Paging

---

- Suppose the virtual and physical memory address spaces are both 32 bits with a 4KB page size.
- 1. Suppose you know that there will only be 4 processes running at the same time, each with a working set size of 256KB, i.e., it uses 256 KB memory most of the time. What is the minimum amount of TLB entries that your system would need to support to be able to map/cache the working set size for one process? What happens if you have more entries? What about if you have fewer entries?
- ANS: A process has a working set size of 256KB which means that the working set fits in  $256\text{KB}/4\text{KB}=64$  pages. This means our TLB should have 64 entries. If you have more entries, then performance will increase since the process often has changing working sets, and it should be able to store more in the TLB. If it has less, then it can't easily translate the addresses in the working set and performance will suffer.
- 2. Suppose you run some benchmarks on the system and you see that the system is utilizing over 99% of its paging disk IO capacity, but only 10% of its CPU. What is a combination of the disk space and memory size that can cause this to occur? Assume you have TLB entries equal to the answer from the previous part.
- ANS: The system is thrashing since there isn't enough memory for the benchmark to run without the system page faulting and having to page in new pages. Since there will be 4 processes that have a working set of 256 KB each, swapping will occur as long as the physical memory size is under 1 MB. This happens regardless of the number of TLB entries and disk size. If the physical memory size is lower than the aggregate working set sizes, thrashing is likely to occur.
- 3. Among 1) increasing TLB size, 2) adding more disk space, and 3) adding more memory, which one would lead to the largest performance increase and why?
- ANS: We should add more memory so that we won't need to page in new pages as often.

---

---

---

---

## Q2. LRU

- In a paging system with page replacement strategy LRU (least recently used), a process performs accesses to memory pages in the given reference sequence:
- Reference sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 1, 2, 0
- The physical memory size is three page frames. Fill in the table for the virtual memory page that is paged into the respective page frame at each given request in the reference sequence.
- Page replacement Algorithms | LRU | Example | OS | Lec-27 | Bhanu Priya
  - [https://www.youtube.com/watch?v=u23ROrISK\\_g](https://www.youtube.com/watch?v=u23ROrISK_g)

| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 1 | 2 | 0 |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| F1  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| F2  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| F3  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

ANS:

---

---

|     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 1 | 2 | 0 |
| F1  | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 0 | 0 |
| F2  |   | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 1 | 1 | 1 |
| F3  |   |   | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 3 | 3 | 3 | 3 | 2 | 2 |

FIFO: 12 page faults

|     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ref | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 1 | 2 | 0 |
| F1  | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 2 | 2 |
| F2  |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 0 |
| F3  |   |   | 1 | 1 | 1 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 1 | 1 | 1 |

LRU: 12 page faults

FIFO: 12 page faults

OPT: 8 page faults

# LRU

- <https://www.youtube.com/watch?v=4wVn97-110r0>

## LRU

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 |
| 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 1 | 1 |
|   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 |
|   |   | 1 | 1 | 1 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| F | F | F | F |   | F |   | F | F | F | F |   |   | F |   |

# fork Example

---

```
void fork1() {  
    int x = 1;  
    pid_t fork_ret = fork();  
    if (fork_ret == 0)  
        printf("Child has x = %d\n", ++x);  
    else  
        printf("Parent has x = %d\n", --x);  
    printf("Bye from process %d with x = %d\n", getpid(), x);  
}
```

- Both **parent** and **child** start/continue execution after fork
- **Child** gets a *copy* of **parent's** data - both processes start with  $x = 1$ 
  - Subsequent changes to  $x$  are **independent**
- Shared open files - `stdout` is the same for both
- **Can't predict execution order** of **parent** and **child** - up to the OS!

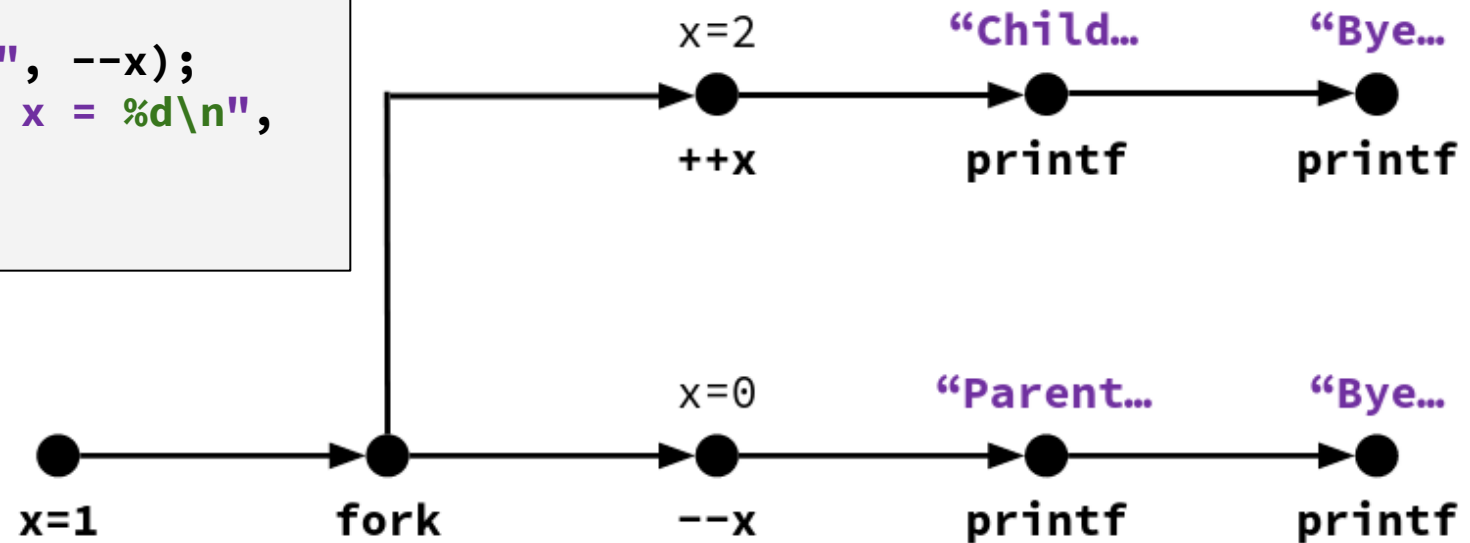
# Modeling fork with Process Graphs

---

- A **process graph** is a useful tool for capturing the partial ordering of statements in a concurrent program
  - Each vertex is the execution of a statement
  - $a \rightarrow b$  means a happens before b
  - Edges can be labeled with current value of variables
  - `printf` vertices can be labeled with output
  - Each graph begins with a vertex with no in-edges
- Any *topological sort* of the graph corresponds to a feasible total ordering
  - Total ordering of vertices where all edges point from left to right

# fork Example: Possible Output

```
void fork1() {  
    int x = 1;  
    pid_t fork_ret = fork();  
    if (fork_ret == 0)  
        printf("Child has x = %d\n", ++x);  
    else  
        printf("Parent has x = %d\n", --x);  
    printf("Bye from process %d with x = %d\n",  
          getpid(), x);  
}
```



# Polling Question

---

Which of the two sequences of outputs are possible?

```
void nestedfork() {  
    printf("L0\n");  
    if (fork() == 0) {  
        printf("L1\n");  
        if (fork() == 0) {  
            printf("L2\n");  
        }  
    }  
    printf("Bye\n");  
}
```

Seq1

L0

L1

Bye

Bye

Bye

L2

A) No

B) No

C) Yes

D) Yes

Seq2

L0

Bye

L1

L2

Bye

Bye

No

Yes

No

Yes

# Fork-Exec

---

- `fork()` creates a copy of the current process
- `exec*()` replaces the current process' code and address space with the code for a different program
  - Whole family of `exec` calls – see `exec(3)` and `execve(2)`

```
void fork_exec(char* path, char* argv[]) {  
    pid_t fork_ret = fork();  
    if (fork_ret != 0) {  
        printf("Parent: created a child %d\n", fork_ret);  
    } else {  
        printf("Child: about to exec a new program\n");  
        execv(path, argv);  
    }  
    printf("This line printed by parent only!\n");  
}
```

# exec-ing a Program

---

Very high-level diagram of what happens when you run the command “ls” in a Linux shell

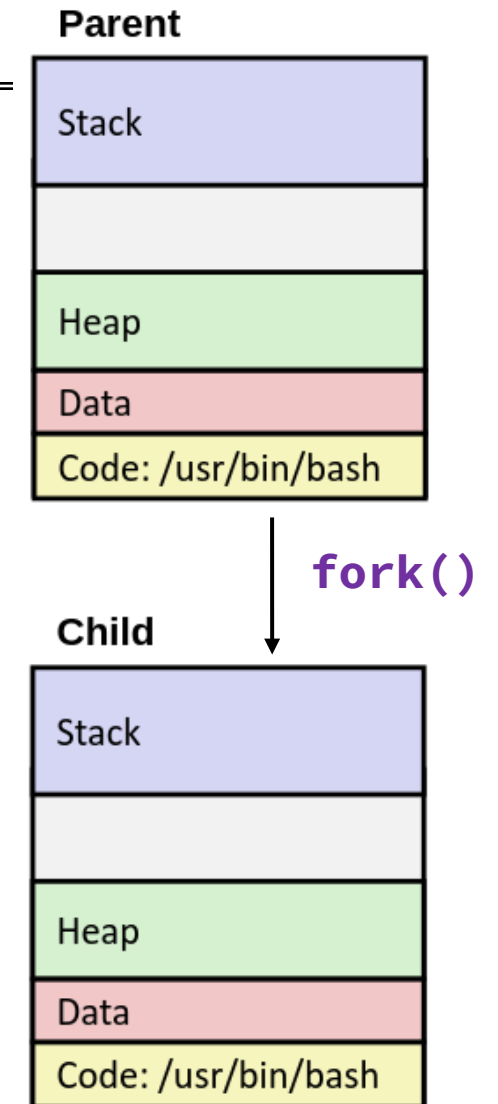
- This is the loading part of CALL!



## exec-ing a Program (pt 2)

Very high-level diagram of what happens when you run the command “ls” in a Linux shell

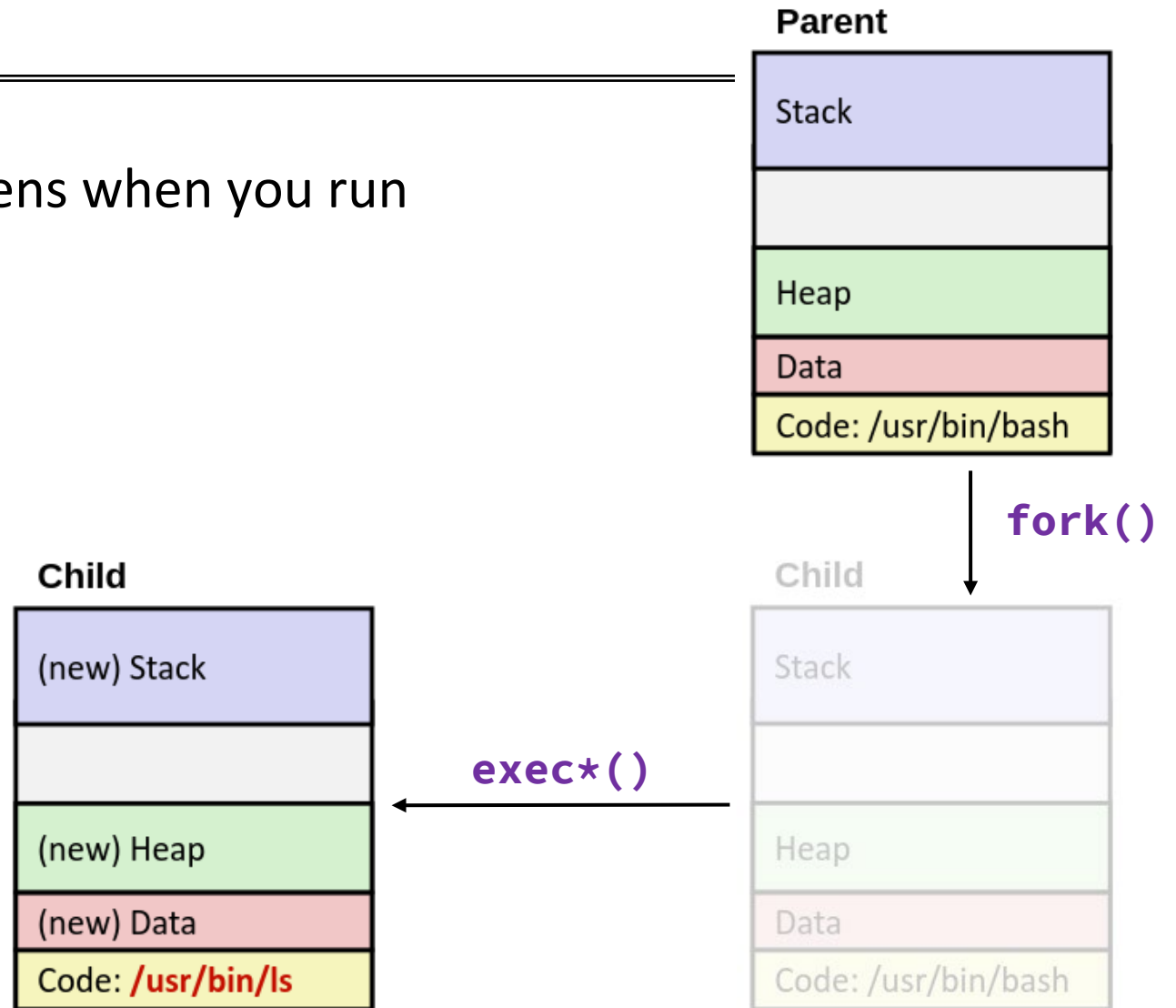
- This is the loading part of CALL!



## exec-ing a Program (pt 3)

Very high-level diagram of what happens when you run the command “ls” in a Linux shell

- This is the loading part of CALL!



## Q4. Schedulability with Shared Resources

---

- Consider the set of 3 periodic tasks scheduled with Rate Monotonic (RM) scheduling, with period, deadline, priority, and WCET parameters given in the table. Tasks 1 and 3 both require semaphore  $s_1$ .
- 1) Calculate priority ceilings of the semaphore  $s_1$ ;
- 2) Determine taskset schedulability for Case (a) no PIP or PCP; Case (b) using PCP

| Task | T=D | C   | Prio | sems  | CS Len | B | R |
|------|-----|-----|------|-------|--------|---|---|
| 1    | 100 | 25  | H    | $s_1$ | 3      |   |   |
| 2    | 200 | 50  | M    | /     | /      |   |   |
| 3    | 300 | 100 | L    | $s_1$ | 30     |   |   |

## Q4. Schedulability with Shared Resources ANS Case (b)

- Ceiling  $C(s_1) = \max(P_1, P_3) = H$
- Blocking times:
  - Task 1:  $B_1 = 30$  (CS length of LP Task 3, associated with semaphore  $s_1$ )
    - » WCRT:  $R_1 = C_1 + B_1 = 25 + 30 = 55 \leq D_1 = 100$
  - Task 2:  $B_2 = 30$  (CS length of LP Task 3, associated with semaphore  $s_1$ )
    - » WCRT:  $R_2 = C_2 + B_2 + \left\lceil \frac{R_2}{T_1} \right\rceil C_1 = 50 + 30 + \left\lceil \frac{R_2}{100} \right\rceil 25 = 130 \leq D_2 = 200$
  - Task 3:  $B_3 = 0$  (Task 3 is the lowest priority task, so it does not experience blocking)
    - » WCRT:  $R_3 = C_3 + \left\lceil \frac{R_3}{T_1} \right\rceil C_1 + \left\lceil \frac{R_3}{T_2} \right\rceil C_2 = 100 + \left\lceil \frac{R_3}{100} \right\rceil 25 + \left\lceil \frac{R_3}{200} \right\rceil 50 = 200 \leq D_3 = 300$
- The taskset is schedulable with shared resources under PCP

| Task | T=D | C   | Prio | sems  | CS Len | B  | R   |
|------|-----|-----|------|-------|--------|----|-----|
| 1    | 100 | 25  | H    | $s_1$ | 3      | 30 | 55  |
| 2    | 200 | 50  | M    | /     | /      | 30 | 130 |
| 3    | 300 | 100 | L    | $s_1$ | 30     | 0  | 200 |

# PCP Example II

Tasks (smaller number denotes higher priority)

| Task Name | $T$  | $D$  | $C$ | priority |
|-----------|------|------|-----|----------|
| A         | 250  | 50   | 14  | 1        |
| B         | 500  | 200  | 50  | 2        |
| C         | 800  | 400  | 90  | 3        |
| D         | 800  | 800  | 20  | 4        |
| E         | 1000 | 1000 | 50  | 5        |
| F         | 2000 | 2000 | 10  | 6        |
| G         | 2000 | 2000 | 10  | 7        |
| H         | 2000 | 2000 | 30  | 8        |

1) LP tasks  
E, F, G, H

Semaphores

| Semaphore | Locked by | Time Held |
|-----------|-----------|-----------|
| $s_2$     | D         | 3         |
| $s_4$     | D         | 3         |
| $s_1$     | D         | 9         |
| $s_2$     | H         | 13        |
| $s_3$     | E         | 4         |
| $s_3$     | B         | 4         |
| $s_4$     | A         | 1         |
| $s_5$     | F         | 7         |
| $s_5$     | H         | 7         |

2)  $s_2, s_3, s_5$   
required by  
LP tasks E,  
F, G, H

| Semaphore | Ceiling |
|-----------|---------|
| $s_1$     | 4       |
| $s_2$     | 4       |
| $s_3$     | 2       |
| $s_4$     | 1       |
| $s_5$     | 6       |

Semaphore Ceilings

Task D has  
blocking time  
 $\max(13, 4) = 13$ ,  
and WCRT=187

| Task Name | $B$ | $R$ |
|-----------|-----|-----|
| A         | 3   | 17  |
| B         | 4   | 68  |
| C         | 4   | 158 |
| D         | 13  | 187 |
| E         | 13  | 237 |
| F         | 13  | 247 |
| G         | 13  | 271 |
| H         | 0   | 288 |

3)  $s_2, s_3$  have  
ceilings with  
higher or equal  
prio than task D

## Blocking Time of Task D

---

- Obtaining D's blocking time
  - Tasks E, F, G, H have lower priority than D (priority number larger than  $\text{prio}(D)=4$ ), and they require semaphores  $s_2$ ,  $s_3$  and  $s_5$
  - $s_2$  and  $s_3$  have ceilings  $\geq \text{prio}(D)$
  - Ceiling  $C(s_2)=\max(\text{prio})$ ,  $C(s_3)$  are higher or equal priority than task D
  - $s_2$  is required by task H for 13 ms;  $s_3$  is required by task E for 4 ms (note that task B has higher priority than D, so it is not included)
  - Blocking time of D =  $\text{MAX}(13, 4) = 13$
- Other tasks' blocking times can be obtained similarly.

## Exam TODO

---

- 1. Multithreaded question, put each thread in a separate box, otherwise students may mistake it as a single threaded program.2.

# Multiple choice questions for Final Exam

---

- \* What operations does NOT move a process to the ready state?
- A) A process is launched/newly admitted into the system.
- B) A process is pre-empted/interrupted by another process.
- C) A process just issued an I/O request and is waiting for the I/O request to be completed.
- D) A process issued an I/O request and the I/O request is just completed.
  
- ANS: C
  
- \* Starvation in scheduling means that some processes may not get scheduled forever. Select the choice that consists of two starvation-free scheduling algorithms.
- A) SJF, SRTF
- B) FIFO, SJF
- C) SJF, RR
- D) FIFO, RR
  
- ANS: D

In slide 23 of lecture 2, how would you implement "wait()" to handle terminated child processes

```
int main() {
    int i;

    for (i = 0; i < 3; i++)
    {fork();}
    printf("Hello\n"); //outside for loop
    return 0;
}
```

Yes but I'm not quite understanding how the closeness of the value to 0 or 1 affects how slow or fast the predicted time updates based on new input

And how that is reflected in the graph

Specifically this

- We can use exponential averaging  $\tau_n = \alpha t_{n-1} + (1-\alpha)\tau_{n-1}$ , where  $t_{n-1}$ ,  $t_{n-2}$ ,  $t_{n-3}$ , etc. are previous CPU burst lengths, and  $\tau_n$  is the predicted next CPU burst length.
  - $t_i$  = actual burst time of process  $P_i$ ,  $i = n, n-1, n-2, \dots$
  - $\tau_n$  = predicted burst time for process  $P_n$
  - $\alpha$  is the smoothing factor ( $0 \leq \alpha \leq 1$ )
    - $\alpha$  large: fast update of  $\tau_n$  based on new input.
    - $\alpha$  small: slow update of  $\tau_n$  based on new input.

# Adding wait()

---

- Consider the following program. None of the parent processes include a wait() call to handle terminated child processes. When any of these child processes terminate, their PCBs remain in the process table as no parent process is waiting on them, resulting in zombie processes after the program finishes execution. Please modify the program by adding wait() calls to prevent zombie processes. (Write your modified program in the empty space below.)

```
int main() {  
    int i;  
    for (i = 0; i < 3; i++)  
    {fork();}  
    printf("Hello\n"); //outside  
    for loop  
    return 0;  
}
```

---

# Fork

---

- Consider the following example program. List all legal outputs this program may produce when executed on a Unix system. The output consists of strings made up of multiple letters.
- `int main() {`
- `printf("A");`
- `int pid = fork();`
- `printf("B");`
- `if (pid) wait(NULL);`
- `printf("C");`
- `}`

# Fork ANS

---

- ANS: Two possible answers: ABBCC and ABCBC
- ABBCC
  - Scenario 1: Parent prints A, forks, prints B, then waits. Child prints B, exits after C. Parent resumes and prints C.
  - Order: A → parent B → child B → child C → parent C.
  - Scenario 2: Parent prints A, forks. Child prints B first, then parent prints B, parent waits, child prints C, and parent prints C.
  - Order: A → child B → parent B → child C → parent C.
- ABCBC
  - Parent prints A, forks. Child runs first, prints B and C before the parent resumes. Parent then prints B, skips waiting (child already exited), and prints C.
  - Order: A → child B → child C → parent B → parent C.
- Wait Synchronization: The parent's wait(NULL) ensures the child's C always appears before the parent's C.

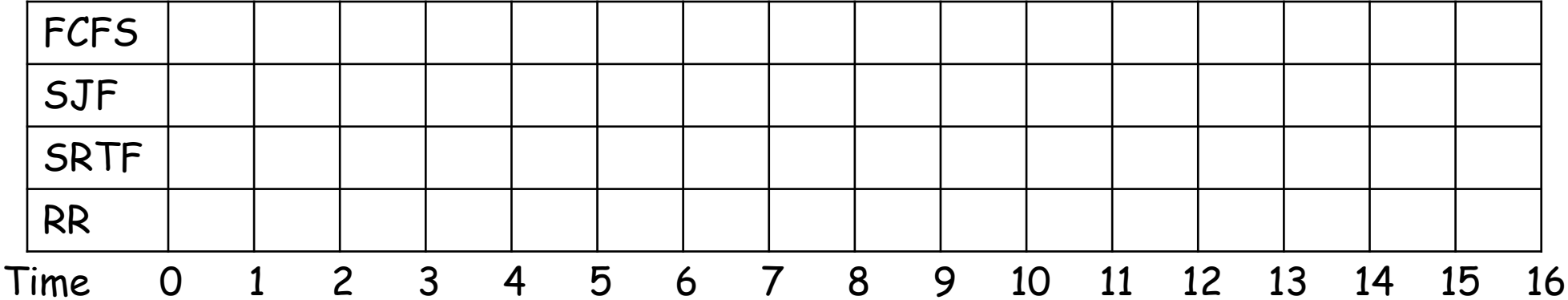
# Scheduling

---

- Here is a table of processes and their arrival and execution times.
- 1) Fill in the scheduling table with the Process ID (PID) that runs at each time instant, under 4 policies: First Come First Serve (FCFS), Shortest Job First (SJF), Shortest-Remaining-Time-First (SRTF), Round-Robin (RR) with timeslice quantum = 1. Assume that context switch overhead is 0 and that new RR processes are added to the head of the queue and new FCFS processes are added to the tail of the queue.
- 2) Compute the finish times and response times for all 5 processes, and the average response time. (If the division is hard, write a fraction like  $28/5$  instead of 5.6)

# Scheduling

| P<br>I<br>D | Arriv.<br>time | Exec<br>Time | FCFS<br>Finish<br>Time | FCFS<br>Response<br>Time | SJF<br>Finish<br>Time | SJF<br>Respons<br>e Time | SRTF<br>Finish<br>Time | SRTF<br>Respons<br>e Time | RR<br>Finish<br>Time | RR<br>Respons<br>e Time |
|-------------|----------------|--------------|------------------------|--------------------------|-----------------------|--------------------------|------------------------|---------------------------|----------------------|-------------------------|
| 1           | 0              | 2            |                        |                          |                       |                          |                        |                           |                      |                         |
| 2           | 1              | 6            |                        |                          |                       |                          |                        |                           |                      |                         |
| 3           | 4              | 1            |                        |                          |                       |                          |                        |                           |                      |                         |
| 4           | 7              | 4            |                        |                          |                       |                          |                        |                           |                      |                         |
| 5           | 8              | 3            |                        |                          |                       |                          |                        |                           |                      |                         |
|             |                |              |                        | Avg RT                   |                       | Avg RT                   |                        | Avg RT                    |                      | Avg RT                  |



Gantt Chart

# Scheduling

| P<br>I<br>D | Arriv.<br>time | Exec<br>Time | FCFS<br>Finish<br>Time | FCFS<br>Response<br>Time | SJF<br>Finish<br>Time | SJF<br>Respons<br>e Time | SRTF<br>Finish<br>Time | SRTF<br>Respons<br>e Time | RR<br>Finish<br>Time | RR<br>Respons<br>e Time |
|-------------|----------------|--------------|------------------------|--------------------------|-----------------------|--------------------------|------------------------|---------------------------|----------------------|-------------------------|
| 1           | 0              | 2            | 2                      | 2                        | 2                     | 2                        | 2                      | 2                         | 3                    | 3                       |
| 2           | 1              | 6            | 8                      | 7                        | 8                     | 7                        | 9                      | 8                         | 13                   | 12                      |
| 3           | 4              | 1            | 9                      | 5                        | 9                     | 5                        | 5                      | 1                         | 5                    | 1                       |
| 4           | 7              | 4            | 13                     | 6                        | 16                    | 9                        | 16                     | 9                         | 16                   | 9                       |
| 5           | 8              | 3            | 16                     | 8                        | 12                    | 4                        | 12                     | 4                         | 15                   | 7                       |
|             |                |              |                        | Avg RT<br>5.6            |                       | Avg RT<br>5.4            |                        | Avg RT<br>4.8             |                      | Avg RT<br>6.4           |

|      |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
|------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| FCFS | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 4 | 4  | 4  | 4  | 5  | 5  | 5  |    |
| SJF  | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 5 | 5  | 5  | 4  | 4  | 4  | 4  |    |
| SRTF | 1 | 1 | 2 | 2 | 3 | 2 | 2 | 2 | 2 | 5 | 5  | 5  | 4  | 4  | 4  | 4  |    |
| RR   | 1 | 2 | 1 | 2 | 3 | 2 | 2 | 4 | 5 | 2 | 4  | 5  | 2  | 4  | 5  | 4  |    |
| Time | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |

Gantt Chart

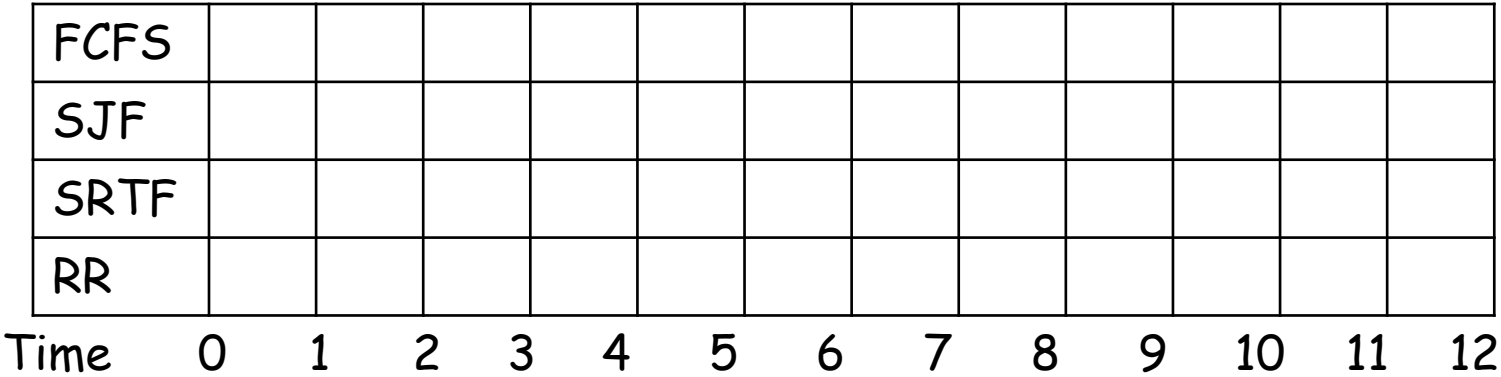
# Scheduling

---

- Here is a table of processes and their arrival and execution times.
- 1) Fill in the scheduling table with the Process ID (PID) that runs at each time instant, under 4 policies: First Come First Serve (FCFS), Shortest Job First (SJF), Shortest-Remaining-Time-First (SRTF), Round-Robin (RR) with timeslice quantum = 1. Assume that context switch overhead is 0 and that new RR processes are added to the head of the queue and new FCFS processes are added to the tail of the queue.
- 2) Compute the finish times and response times for all 5 processes, and the average response time. (If the division is hard, write a fraction like  $28/5$  instead of 5.6)

# Scheduling

| P<br>I<br>D | Arriv.<br>time | Exec<br>Time | FCFS<br>Finish<br>Time | FCFS<br>Response<br>Time | SJF<br>Finish<br>Time | SJF<br>Response<br>Time | SRTF<br>Finish<br>Time | SRTF<br>Response<br>Time | RR<br>Finish<br>Time | RR<br>Response<br>Time |
|-------------|----------------|--------------|------------------------|--------------------------|-----------------------|-------------------------|------------------------|--------------------------|----------------------|------------------------|
| 1           | 0              | 3            |                        |                          |                       |                         |                        |                          |                      |                        |
| 2           | 1              | 5            |                        |                          |                       |                         |                        |                          |                      |                        |
| 3           | 3              | 2            |                        |                          |                       |                         |                        |                          |                      |                        |
| 4           | 9              | 2            |                        |                          |                       |                         |                        |                          |                      |                        |
|             |                |              |                        | Avg RT                   |                       | Avg RT                  |                        | Avg RT                   |                      | Avg RT                 |



Gantt Chart

# Scheduling

| P<br>I<br>D | Arriv.<br>time | Exec<br>Time | FCFS<br>Finish<br>Time | FCFS<br>Response<br>Time | SJF<br>Finish<br>Time | SJF<br>Response<br>Time | SRTF<br>Finish<br>Time | SRTF<br>Response<br>Time | RR<br>Finish<br>Time | RR<br>Response<br>Time |
|-------------|----------------|--------------|------------------------|--------------------------|-----------------------|-------------------------|------------------------|--------------------------|----------------------|------------------------|
| 1           | 0              | 3            | 3                      | 3                        | 3                     | 3                       | 3                      | 3                        | 6                    | 6                      |
| 2           | 1              | 5            | 8                      | 7                        | 10                    | 9                       | 10                     | 9                        | 11                   | 10                     |
| 3           | 3              | 2            | 10                     | 7                        | 5                     | 2                       | 5                      | 2                        | 7                    | 4                      |
| 4           | 9              | 2            | 12                     | 3                        | 12                    | 3                       | 12                     | 3                        | 12                   | 3                      |
|             |                |              |                        | Avg RT<br>5              |                       | Avg RT<br>4.25          |                        | Avg RT<br>4.25           |                      | Avg RT<br>5.75         |

|      |   |   |   |   |   |   |   |   |   |   |    |    |    |
|------|---|---|---|---|---|---|---|---|---|---|----|----|----|
| FCFS | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 4  | 4  |    |
| SJF  | 1 | 1 | 1 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 4  | 4  |    |
| SRTF | 1 | 1 | 1 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 4  | 4  |    |
| RR   | 1 | 2 | 1 | 3 | 2 | 1 | 3 | 2 | 2 | 4 | 2  | 4  |    |
| Time | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |

Gantt Chart

c. (15 points) Consider the following processes, arrival times, and CPU processing requirements:

| Process Name | Arrival Time | Processing Time |
|--------------|--------------|-----------------|
| 1            | 0            | 3               |
| 2            | 1            | 5               |
| 3            | 3            | 2               |
| 4            | 9            | 2               |

For each of the following scheduling algorithms, fill in the table with the process that is running on the CPU (for timeslice-based algorithms, assume a 1 unit timeslice):  
For RR, assume that an arriving thread is scheduled to run at the beginning of its arrival time.

| Time                  | FIFO            | RR                  | SRTCF              |
|-----------------------|-----------------|---------------------|--------------------|
| 0                     | 1               | 1                   | 1                  |
| 1                     | 1               | 2                   | 1                  |
| 2                     | 1               | 1                   | 1                  |
| 3                     | 2               | 3                   | 3                  |
| 4                     | 2               | 2                   | 3                  |
| 5                     | 2               | 1                   | 2                  |
| 6                     | 2               | 3                   | 2                  |
| 7                     | 2               | 2                   | 2                  |
| 8                     | 3               | 2                   | 2                  |
| 9                     | 3               | 4                   | 2                  |
| 10                    | 4               | 2                   | 4                  |
| 11                    | 4               | 4                   | 4                  |
| Average response time | $3+7+7+3/4 = 5$ | $6+10+4+3/4 = 5.75$ | $3+2+9+3/4 = 4.25$ |

Each column is worth 5 points: 3 for correctness of the schedule (we deducted 1/2/3 points if you made minor/intermediate/major mistakes), and 2 for the average response time (1 point was deducted for minor errors).

| Time                  | FIFO            | RR                  | SRTCF              |
|-----------------------|-----------------|---------------------|--------------------|
| 0                     | 1               | 1                   | 1                  |
| 1                     | 1               | 2                   | 1                  |
| 2                     | 1               | 1                   | 1                  |
| 3                     | 2               | 3                   | 3                  |
| 4                     | 2               | 2                   | 3                  |
| 5                     | 2               | 1                   | 2                  |
| 6                     | 2               | 3                   | 2                  |
| 7                     | 2               | 2                   | 2                  |
| 8                     | 3               | 2                   | 2                  |
| 9                     | 3               | 4                   | 2                  |
| 10                    | 4               | 2                   | 4                  |
| 11                    | 4               | 4                   | 4                  |
| Average response time | $3+7+7+3/4 = 5$ | $6+10+4+3/4 = 5.75$ | $3+2+9+3/4 = 4.25$ |

- 
- Modify this program by adding or removing semaphore operations to ensure that after both threads finish execution,
  - (1)  $x = 0$
  - (2) Or,  $x=1$
  - (3) Or,  $x=3$

if you wanted to guarantee  $x=3$ , how would this be modified?

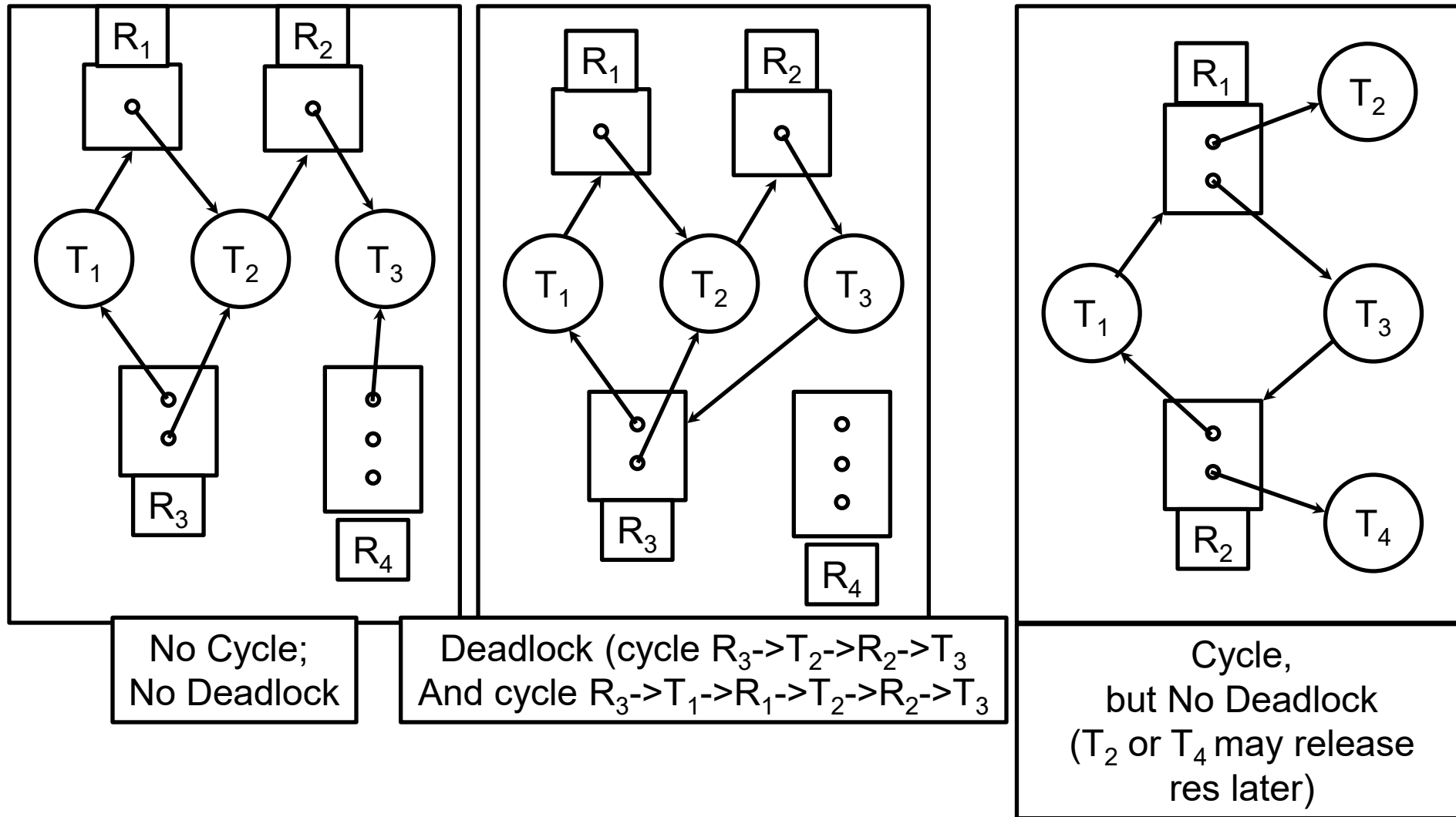
```
int y=0, z=0;  
semaphore s = 1;
```

```
1 t1() {  
2     int x;  
3     s.wait();  
4     x = y + z;  
5     s.signal();  
6 }
```

```
1 t2() {  
2     s.wait();  
3     y = 1;  
4     z = 2;  
5     s.signal();  
6 }
```

---

# Resource Allocation Graph Examples



# (1) No Deadlock Morning Exam

Max

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 1  | 0  |
| T2 | 1  | 1  | 1  | 0  |
| T3 | 0  | 1  | 0  | 1  |

Allocation

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 0  | 0  | 1  | 0  |
| T2 | 1  | 0  | 1  | 0  |
| T3 | 0  | 1  | 0  | 1  |

Need

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 0  | 0  |
| T2 | 0  | 1  | 0  | 0  |
| T3 | 0  | 0  | 0  | 0  |

Total

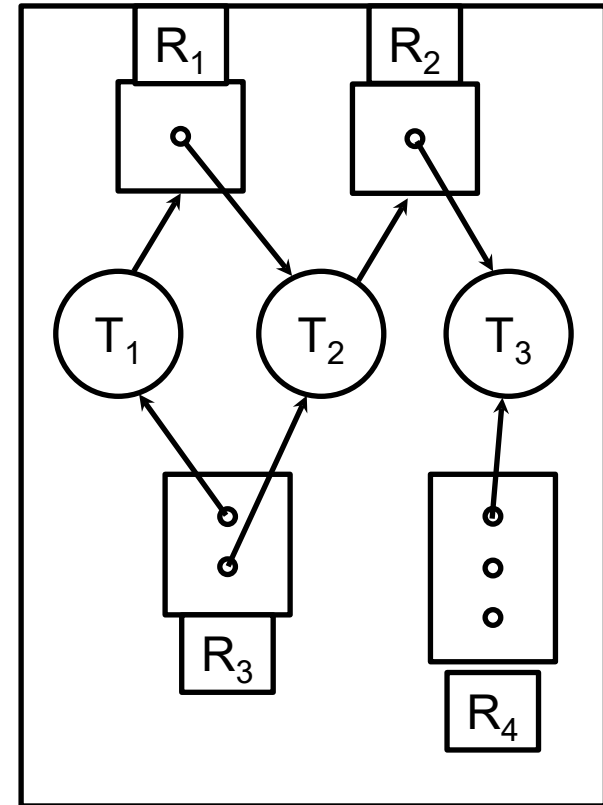
| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 1  | 1  | 2  | 3  |

Available

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 0  | 0  | 0  | 2  |

Available resources after completion of each process

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
|    | 0  | 0  | 0  | 2  |
| T3 |    |    |    |    |
| T2 |    |    |    |    |
| T1 |    |    |    |    |



Run Banker's algorithm to check if the current state is safe. If yes, give a safe sequence of process completions and fill in the table with the sequence of process completions without deadlock, and available resources after the completion of each process.

# (1) No Deadlock ANS Morning Exam

Max

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 1  | 0  |
| T2 | 1  | 1  | 1  | 0  |
| T3 | 0  | 1  | 0  | 1  |

Allocation

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 0  | 0  | 1  | 0  |
| T2 | 1  | 0  | 1  | 0  |
| T3 | 0  | 1  | 0  | 1  |

Need

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 0  | 0  |
| T2 | 0  | 1  | 0  | 0  |
| T3 | 0  | 0  | 0  | 0  |

Total

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 1  | 1  | 2  | 3  |

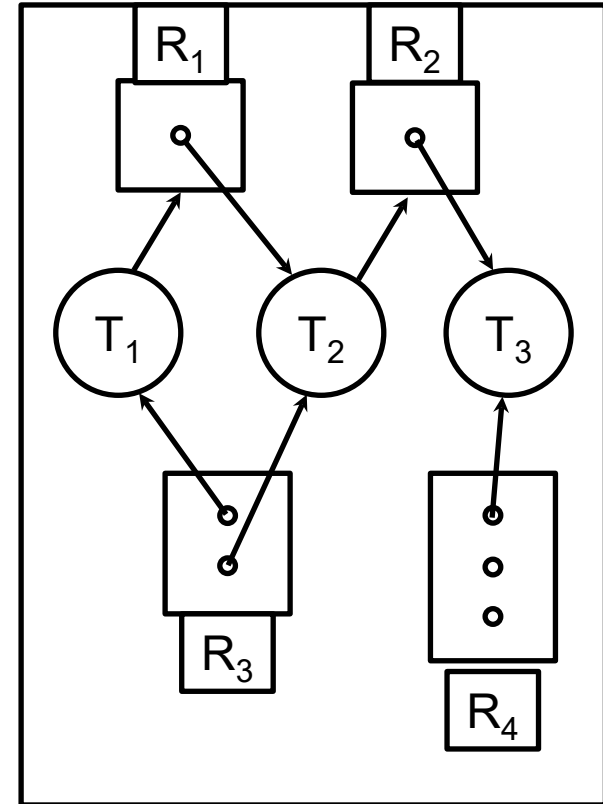
Available

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 0  | 0  | 0  | 2  |

Available

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
|    | 0  | 0  | 0  | 2  |
| T3 | 0  | 1  | 0  | 3  |
| T2 | 1  | 1  | 1  | 3  |
| T1 | 1  | 1  | 2  | 3  |

Safe Sequence: T3, T2, T1



# (1) Deadlock ANS Morning Exam

Max

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 1  | 0  |
| T2 | 1  | 1  | 1  | 0  |
| T3 | 0  | 1  | 1  | 0  |

Allocation

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 0  | 0  | 1  | 0  |
| T2 | 1  | 0  | 1  | 0  |
| T3 | 0  | 1  | 0  | 0  |

Need

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 0  | 0  |
| T2 | 0  | 1  | 0  | 0  |
| T3 | 0  | 0  | 1  | 0  |

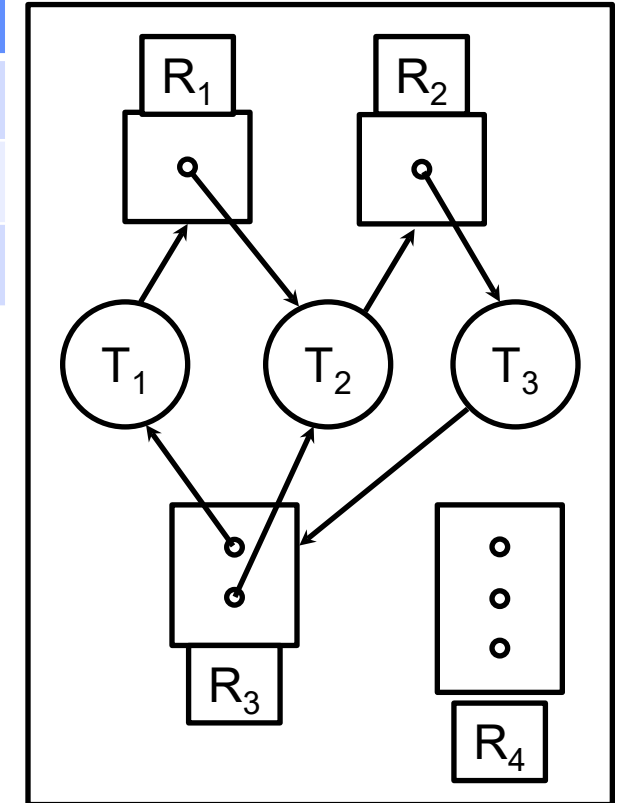
Total

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 1  | 1  | 2  | 3  |

Available

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 0  | 0  | 0  | 3  |

Deadlock, no safe sequence



# (1) Deadlock Evening Exam

Max

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 1  | 0  |
| T2 | 1  | 0  | 1  | 0  |
| T3 | 0  | 1  | 1  | 0  |

Allocation

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 0  | 0  | 1  | 0  |
| T2 | 1  | 0  | 1  | 0  |
| T3 | 0  | 1  | 0  | 0  |

Need

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
| T1 | 1  | 0  | 0  | 0  |
| T2 | 0  | 1  | 0  | 0  |
| T3 | 0  | 0  | 1  | 0  |

Total

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 1  | 1  | 2  | 3  |

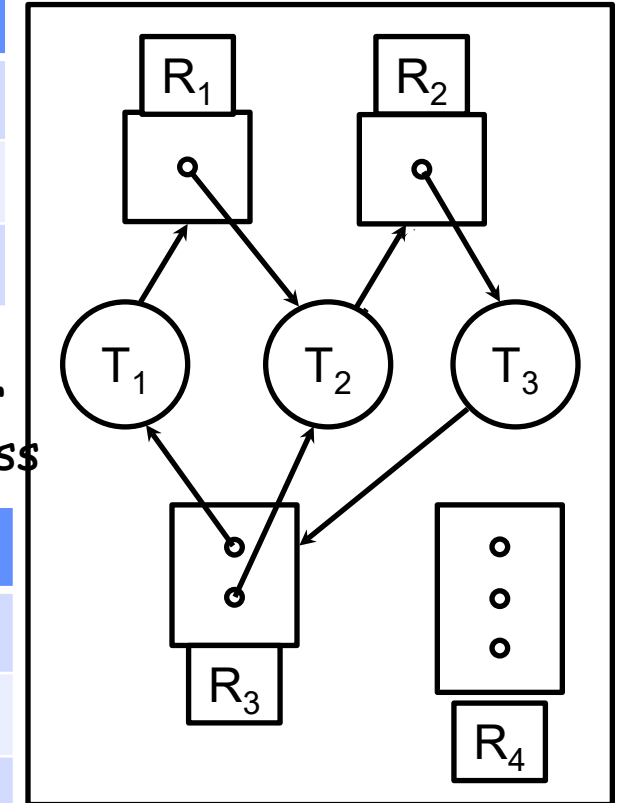
Available

| R1 | R2 | R3 | R4 |
|----|----|----|----|
| 0  | 0  | 0  | 3  |

Available resources after completion of each process

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
|    | 0  | 0  | 0  | 3  |
| T2 | 1  | 0  | 1  | 3  |
| T1 | 1  | 0  | 2  | 3  |
| T3 | 1  | 1  | 2  | 3  |

|    | R1 | R2 | R3 | R4 |
|----|----|----|----|----|
|    | 0  | 0  | 0  | 3  |
| T2 | 1  | 0  | 1  | 3  |
| T3 | 1  | 1  | 1  | 3  |
| T1 | 1  | 1  | 2  | 3  |



Safe sequence T2, T1, T3 or T2, T3, T1

# Exam Quiz 1: Dining Lawyers

---

- Consider the Dining Lawyers problem. There are 3 lawyers P1 to P3. Each lawyer has 4 arms, and there is a pile of knives and forks at center of the table. Each lawyer follows the following steps:
  - (1) Pick up a knife
  - (2) Pick up another knife
  - (3) Pick up a fork
  - (4) Pick up another fork
  - (5) Eat
  - (6) Return the knife and fork to the pile
- Use Banker's algorithm to check if it is possible for the system to get into a deadlock.
- 1) The pile contains 3 knives and 2 forks. (Yes deadlock on knives)
- 2) The pile contains 4 knives and 2 forks. (Yes deadlock on forks)
- 3) The pile contains 4 knives and 3 forks. (No deadlock)
- 4) The pile contains 4 knives and 3 forks. (No deadlock)
- For each question, please write out the initial state encoded in 3 matrices Max, Allocation, Need, and 2 vectors Total and Available. If there is no possible deadlock, explain why. If there is a (potential) deadlock, show an unsafe system state and use Banker's algorithm to check it.

## Quiz 2: Dining Lawyers Solution: 3 lawyers, each with 4 arms, 3 knives and 2 forks

Initially, all knives and forks are free.

| Max                                                                                                    | Allocation |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
|--------------------------------------------------------------------------------------------------------|------------|---|----------------------------------------------|---|---|---|--------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|
| <table><tr><td>2</td><td>2</td></tr><tr><td>2</td><td>2</td></tr><tr><td>2</td><td>2</td></tr></table> | 2          | 2 | 2                                            | 2 | 2 | 2 | <table><tr><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td></tr></table> | 0 | 0 | 0 | 0 | 0 | 0 |
| 2                                                                                                      | 2          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 2                                                                                                      | 2          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 2                                                                                                      | 2          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 0                                                                                                      | 0          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 0                                                                                                      | 0          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 0                                                                                                      | 0          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| Total                                                                                                  | Available  |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| <table><tr><td>3</td><td>2</td></tr></table>                                                           | 3          | 2 | <table><tr><td>3</td><td>2</td></tr></table> | 3 | 2 |   |                                                                                                        |   |   |   |   |   |   |
| 3                                                                                                      | 2          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 3                                                                                                      | 2          |   |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |

P1, P2 and P3 each grab 1 knife.

| Max                                                                                                    | Allocation | Need |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
|--------------------------------------------------------------------------------------------------------|------------|------|----------------------------------------------|---|---|---|--------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|--------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|
| <table><tr><td>2</td><td>2</td></tr><tr><td>2</td><td>2</td></tr><tr><td>2</td><td>2</td></tr></table> | 2          | 2    | 2                                            | 2 | 2 | 2 | <table><tr><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td></tr></table> | 1 | 0 | 1 | 0 | 1 | 0 | <table><tr><td>1</td><td>2</td></tr><tr><td>1</td><td>2</td></tr><tr><td>1</td><td>2</td></tr></table> | 1 | 2 | 1 | 2 | 1 | 2 |
| 2                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 2                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 2                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 0          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 0          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 0          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 1                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| Total                                                                                                  | Available  |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| <table><tr><td>3</td><td>2</td></tr></table>                                                           | 3          | 2    | <table><tr><td>0</td><td>2</td></tr></table> | 0 | 2 |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 3                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |
| 0                                                                                                      | 2          |      |                                              |   |   |   |                                                                                                        |   |   |   |   |   |   |                                                                                                        |   |   |   |   |   |   |

Available resources after completion of each process

|          | K | F |
|----------|---|---|
| Init     | 0 | 2 |
| Deadlock |   |   |

Current state is deadlocked on knives.

It is not possible for the lawyers to be deadlocked on forks.

Quiz 2: Dining Lawyers Solution: 3 lawyers, each with 4 arms, 4 knives and 2 forks

Initially, all knives and forks are free.

| Max   |   | Allocation |   |
|-------|---|------------|---|
| 2     | 2 | 0          | 0 |
| 2     | 2 | 0          | 0 |
| 2     | 2 | 0          | 0 |
| Total |   | Available  |   |
| 4     | 2 | 4          | 2 |

P1 grabs 2 knives and 1 fork; P2 grabs 2 knives and 1 fork.

| Max   |   | Allocation |   | Need |   |
|-------|---|------------|---|------|---|
| 2     | 2 | 2          | 1 | 0    | 1 |
| 2     | 2 | 2          | 1 | 0    | 1 |
| 2     | 2 | 0          | 0 | 2    | 2 |
| Total |   | Available  |   |      |   |
| 4     | 2 | 0          | 0 |      |   |

Available resources after completion of each process

|          | K | F |
|----------|---|---|
| Init     | 0 | 0 |
| Deadlock |   |   |

Current state is deadlocked on forks.

It is not possible for the lawyers to be deadlocked on knives, but possible to deadlock on forks

Quiz 2: Dining Lawyers Solution: 3 lawyers, each with 4 arms, 4 knives and 3 forks

Initially, all  
knives and  
forks are free.

| Max   |   | Allocation |   |
|-------|---|------------|---|
| 2     | 2 | 0          | 0 |
| 2     | 2 | 0          | 0 |
| 2     | 2 | 0          | 0 |
| Total |   | Available  |   |
| 4     | 3 | 4          | 3 |

Not possible to get  
into deadlock

| Max   |   | Allocation |   | Need |   |
|-------|---|------------|---|------|---|
| 2     | 2 | 2          | 1 | 0    | 1 |
| 2     | 2 | 2          | 2 | 0    | 0 |
| 2     | 2 | 0          | 0 | 2    | 2 |
| Total |   | Available  |   |      |   |
| 4     | 3 | 0          | 0 |      |   |

Available resources after  
completion of each process

|      | K | F |
|------|---|---|
| Init | 4 | 2 |
| P1   | 1 |   |
| P2   |   |   |

It is not possible for the  
lawyers to be deadlocked  
on either knives or forks

# Exam Quiz 1: Dining Lawyers

---

- Consider the Dining Lawyers problem. There are 2 lawyers P1 to P2. Each lawyer has 4 arms, and there is a pile of knives and forks at center of the table. Each lawyer follows the following steps:
  - (1) Pick up a knife
  - (2) Pick up another knife
  - (3) Pick up a fork
  - (4) Pick up another fork
  - (5) Eat
  - (6) Return the knife and fork to the pile
- Use Banker's algorithm to check if it is possible for the system to get into a deadlock for the following cases:
  - 1) The pile contains 2 knives and 2 forks. (Yes deadlock on knives)
  - 2) The pile contains 3 knives and 2 forks. (No deadlock)
  - 3) The pile contains 4 knives and 2 forks. (Yes deadlock, on forks)
  - 4) The pile contains 4 knives and 3 forks. (No deadlock)
- For each question, please write out the initial state encoded in 3 matrices Max, Allocation, Need, and 2 vectors Total and Available. If there is no possible deadlock, explain why. If there is a (potential) deadlock, show an unsafe system state and use Banker's algorithm to check it.

Quiz 2: Dining Lawyers Solution: 2 lawyers, each with 4 arms, 2 knives and 2 forks

Initially, all  
knives and  
forks are free.

| Max |   | Allocation |   |                   |
|-----|---|------------|---|-------------------|
|     |   |            |   |                   |
| 2   | 2 | 0          | 0 | P1 grabs 1 knife; |
| 2   | 2 | 0          | 0 | P2 grabs 1 knife. |

| Total |   | Available |   |
|-------|---|-----------|---|
| 2     | 2 | 2         | 2 |

| Max |   | Allocation |   | Need |   |
|-----|---|------------|---|------|---|
|     |   |            |   |      |   |
| 2   | 2 | 1          | 0 | 1    | 2 |
| 2   | 2 | 1          | 0 | 1    | 2 |

| Total |   | Available |   |
|-------|---|-----------|---|
| 2     | 2 | 0         | 2 |

Current state is  
deadlocked on knives.

Available resources after  
completion of each process

|          | K | F |
|----------|---|---|
| Init     | 0 | 2 |
| Deadlock |   |   |

## Quiz 2: Dining Lawyers Solution: 2 lawyers, each with 4 arms, 3 knives and 2 forks

Initially, all knives and forks are free.

Max Allocation

|   |   |   |   |
|---|---|---|---|
| 2 | 2 | 0 | 0 |
| 2 | 2 | 0 | 0 |

P1 grabs 2 knives and then 2 forks;  
P2 grabs 1 knife.

Total Available

|   |   |   |   |
|---|---|---|---|
| 3 | 2 | 3 | 2 |
|---|---|---|---|

Max Allocation Need

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 2 | 2 | 2 | 0 | 0 | 2 |
| 2 | 2 | 1 | 0 | 1 | 2 |

Total Available

|   |   |   |   |
|---|---|---|---|
| 3 | 2 | 0 | 2 |
|---|---|---|---|

Available resources after completion of each process

|      | K | F |
|------|---|---|
| Init | 0 | 2 |
| P1   | 2 | 2 |
| P2   | 3 | 2 |

It is not possible for deadlock.

Quiz 2: Dining Lawyers Solution: 2 lawyers, each with 4 arms, 4 knives and 2 forks

|                                                 |       |   |            |   |                                                                      |
|-------------------------------------------------|-------|---|------------|---|----------------------------------------------------------------------|
|                                                 | Max   |   | Allocation |   |                                                                      |
| Initially, all<br>knives and<br>forks are free. | 2     | 2 | 0          | 0 | P1 grabs 2 knives<br>and 1 fork; P2<br>grabs 2 knives and<br>1 fork. |
|                                                 | 2     | 2 | 0          | 0 |                                                                      |
|                                                 | Total |   | Available  |   |                                                                      |
|                                                 | 4     | 2 | 4          | 2 |                                                                      |

| Max   |   | Allocation |   | Need |   |
|-------|---|------------|---|------|---|
| 2     | 2 | 2          | 1 | 0    | 1 |
| 2     | 2 | 2          | 1 | 0    | 1 |
| Total |   | Available  |   |      |   |
| 4     | 2 | 0          | 0 |      |   |

Available resources after completion of each process

|          | K | F |
|----------|---|---|
| Init     | 0 | 0 |
| Deadlock |   |   |

Current state is deadlocked on forks.

## Quiz 2: Dining Lawyers Solution: 2 lawyers, each with 4 arms, 4 knives and 3 forks

Initially, all knives and forks are free.

Max Allocation

|   |   |   |   |
|---|---|---|---|
| 2 | 2 | 0 | 0 |
| 2 | 2 | 0 | 0 |

P1 grabs 2 knives and 2 forks; P2 grabs 2 knives and 1 fork.

Total Available

|   |   |   |   |
|---|---|---|---|
| 4 | 3 | 4 | 3 |
|---|---|---|---|

Max Allocation Need

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 2 | 2 | 2 | 2 | 0 | 0 |
| 2 | 2 | 2 | 1 | 0 | 1 |

Total Available

|   |   |   |   |
|---|---|---|---|
| 4 | 3 | 0 | 0 |
|---|---|---|---|

Available resources after completion of each process

|      | K | F |
|------|---|---|
| Init | 0 | 0 |
| P1   | 2 | 2 |
| P2   | 4 | 3 |

Current state is safe: P1 can finish first, so it is not possible for deadlock.

---

---

| P<br>I<br>D | Arriv.<br>time | Priority | CPU<br>Burst | IO<br>Burst | CPU<br>Burst |
|-------------|----------------|----------|--------------|-------------|--------------|
| 1           | 0              | 2        | 1            | 5           | 3            |
| 2           | 2              | 1        | 3            | 3           | 1            |
| 3           | 3              | 3        | 2            | 3           | 1            |
|             |                |          | Avg RT       |             | Avg RT       |

# Wait() III

---

```
int main() {
    int i;

    for (i = 0; i < 2; i++) {
        pid_t pid = fork();

        if (pid == 0) {
            // Child process
            printf("Hello from Child %d\n", i);
            return 0; // Exit child process
        } else if (pid > 0) {
            // Parent process
            printf("Hello from Parent %d\n", i);
            wait(NULL); // Wait for immediate child to
            terminate
            printf("Hello from Parent again %d\n", i);
        }

        printf("Parent exiting\n");
        return 0;
    }
}
```

- Output: 4 possible outputs
- 1) These 2 statements interleaved in any order:
  - Hello from Parent 0
  - Hello from Child 0
- 2) Hello from Parent again 0
- 3) These 2 statements interleaved in any order:
  - Hello from Parent 1
  - Hello from Child 1
- 4) Hello from Parent again 1
- Parent exiting

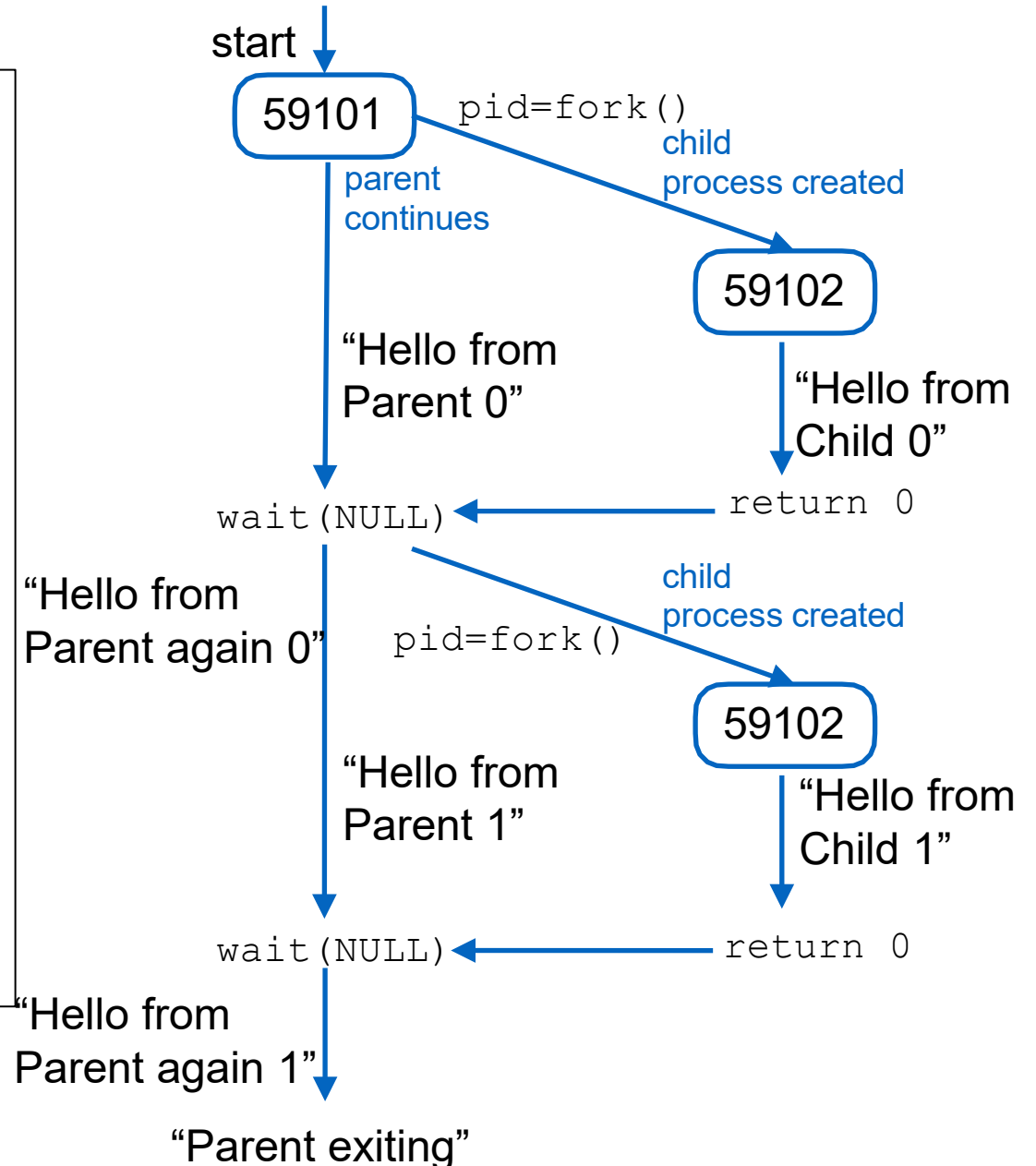
# Wait() III

```
int main() {
    int i;

    for (i = 0; i < 2; i++) {
        pid_t pid = fork();

        if (pid == 0) {
            // Child process
            printf("Hello from Child %d\n", i);
            return 0; // Exit child process
        } else if (pid > 0) {
            // Parent process
            printf("Hello from Parent %d\n", i);
            wait(NULL); // Wait for immediate child to
            terminate
            printf("Hello from Parent again %d\n", i);
        }
    }

    printf("Parent exiting\n");
    return 0;
}
```



# Wait() IV

```
int main() {
    int i;

    for (i = 0; i < 2; i++) {
        pid_t pid = fork();

        if (pid == 0) {
            // Child process
            printf("Hello from Child %d\n", i);
            return 0; // Exit child process
        } else if (pid > 0) {
            // Parent process
            continue;
        }
    }

    // Parent process waits for all child processes to
    terminate
    for (i = 0; i < 2; i++) {
        printf("Hello from Parent %d\n", i);
        wait(NULL); // Wait for a child process to
    terminate
        printf("Hello from Parent again %d\n", i);
    }
    printf("Parent exiting\n");
    return 0;
}
```

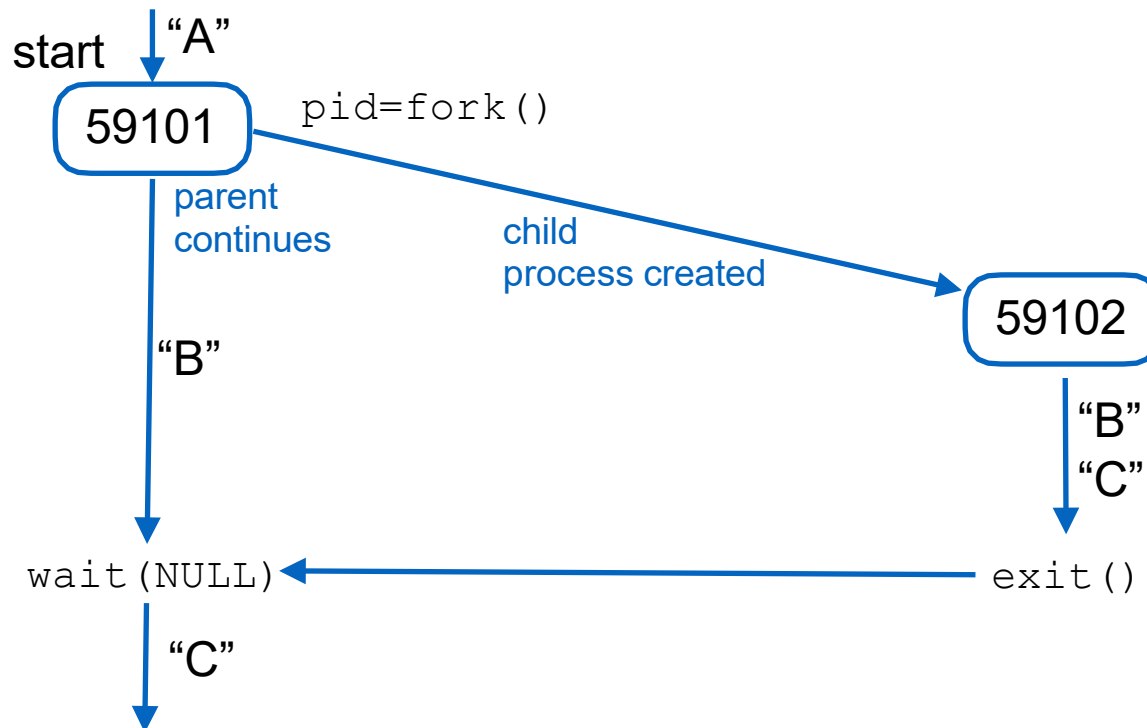
- These statements interleaved in some order:
  - Hello from Parent 0
  - Hello from Child 0
  - Hello from Parent 1
  - Hello from Child 1
  - Hello from Parent again 0
  - Hello from Parent again 1
- With constraints:
  - The parent prints in program order (Parent 0, Parent again 0, Parent 1, Parent again 1). wait(NULL) reaps any one child, so:
  - “Hello from Parent again 0” must appear after at least one of “Hello from Child 0” and “Hello from Child 1”
  - “Hello from Parent again 1” must appear after both “Hello from Child 0” and “Hello from Child 1”
- A final "Parent exiting" message printed last.

## Q2 d) (5 pts) Modify it to distinguish between parent and child

```
int main() {  
    printf("A");  
    int pid = fork();  
    printf("B");  
    if (pid>0) wait(NULL);  
    printf("C");  
}
```

Output:  
ABBCC  
ABCBC

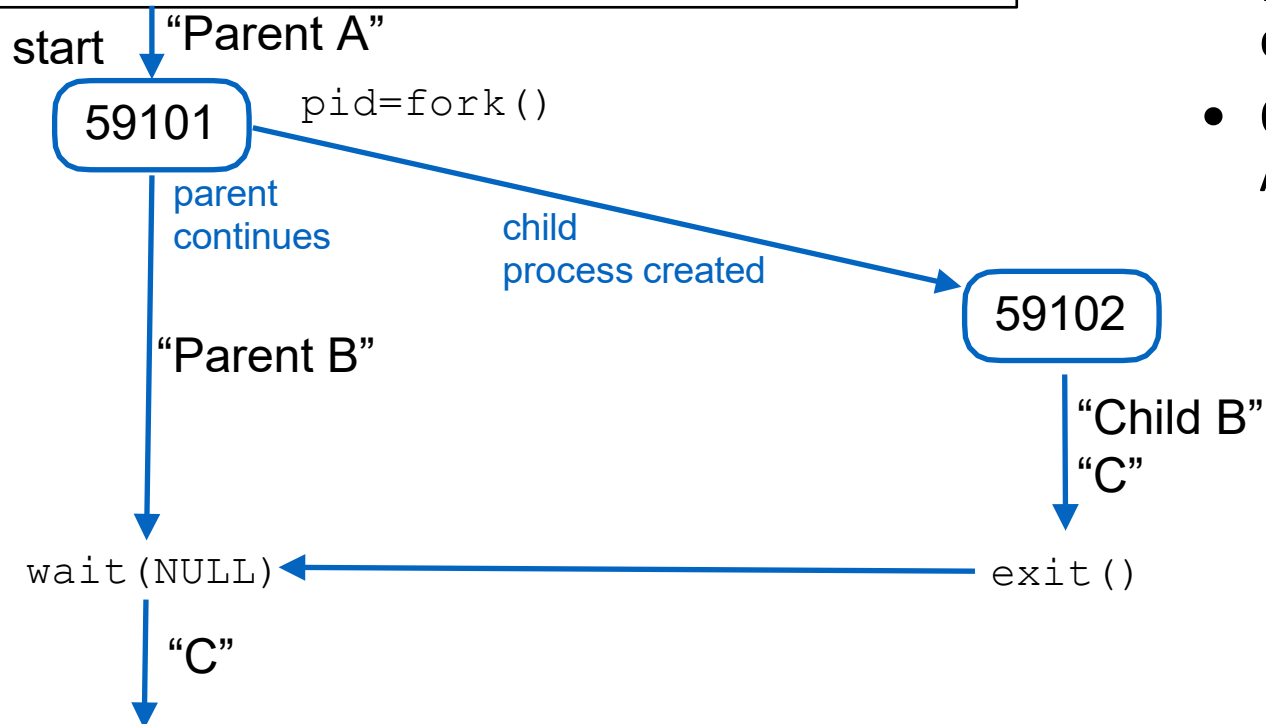
- ANS: ABBCC or ABCBC
- Parent prints A
- fork() is called. Concurrently:
  - Parent prints B, then waits for child to finish
  - Child prints B, then prints C
- Parent continues after child finishes execution, and prints C.
- Output sequence can be either ABBCC and ABCBC
  - Initially parent prints A first
  - Parent's printing of B and child's printing of B, C can interleave in arbitrary order, with two possible outputs BBC or BCB
  - The parent's wait(NULL) ensures that parent prints C last.



## Q2 d) (5 pts) Modify it to distinguish between parent and child

```
int main() {
    printf("Parent A\n");
    int pid = fork();
    if (pid==0){
        printf("Child B\n");}
    else{
        printf("Parent B\n");
        wait(NULL);
    }
    printf("C\n");
}
```

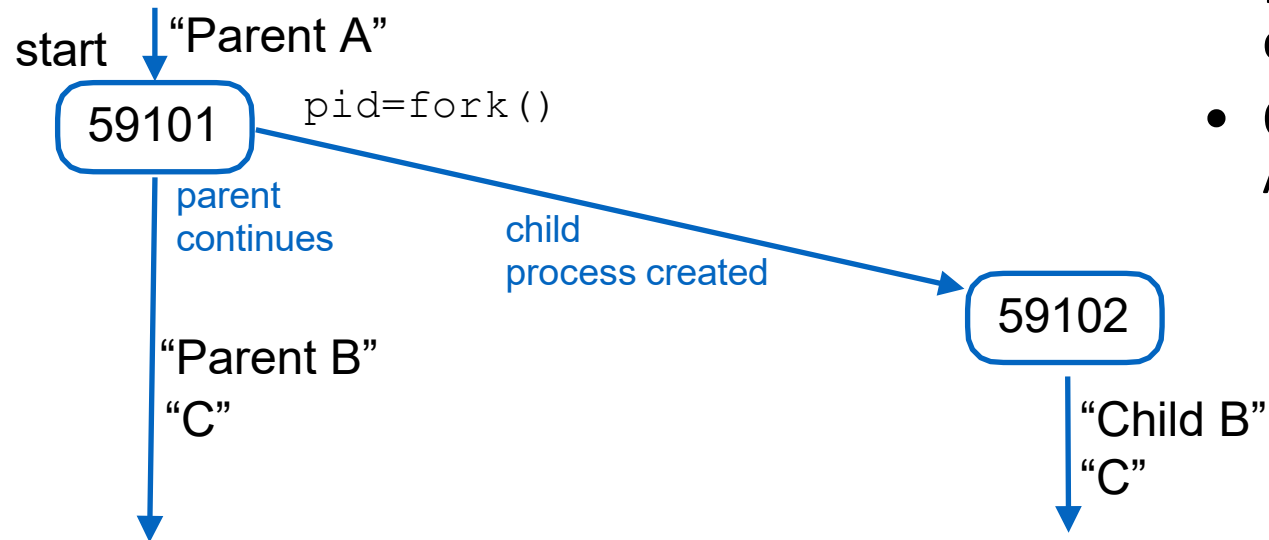
- ANS: Starts with "Parent A", ends with "C". In the middle, any interleaving of "Parent B" with "Child B""C" (3 interleavings)
- Parent prints A
- fork() is called. Concurrently:
  - Parent prints B, then waits for child to finish
  - Child prints B, then prints C
- Parent continues after child finishes execution, and prints C.
- Output sequence can be either ABBCC and ABCBC
  - Initially parent prints A first
  - Parent's printing of B and child's printing of B, C can interleave in arbitrary order, with two possible outputs BBC or BCB
  - The parent's wait(NULL) ensures that parent prints C last.

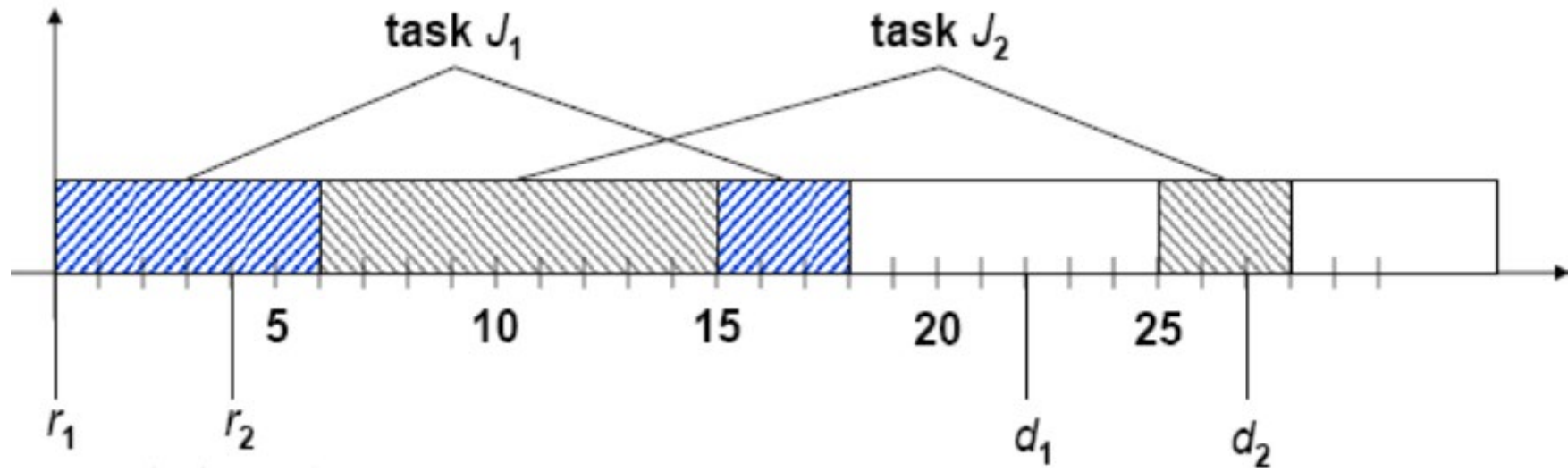


## Q2 d) (5 pts) Modify it to distinguish between parent and child

```
int main() {  
    printf("Parent A\n");  
    int pid = fork();  
    if (pid==0){  
        printf("Child B\n");  
    }  
    else{  
        printf("Parent B\n");  
    }  
    printf("C\n");  
}
```

- ANS: Starts with "Parent A". Followed by any interleaving of "Parent B""C" with "Child B""C" (4 interleavings)
- Parent prints A
- fork() is called. Concurrently:
  - Parent prints B, then waits for child to finish
  - Child prints B, then prints C
- Parent continues after child finishes execution, and prints C.
- Output sequence can be either ABBCC and ABCBC
  - Initially parent prints A first
  - Parent's printing of B and child's printing of B, C can interleave in arbitrary order, with two possible outputs BBC or BCB
  - The parent's wait(NULL) ensures that parent prints C last.





- Given:
  - WCET  $C_1 = 9, C_2 = 12$  (CPU burst 9, IO burst)
  - Relative Deadline:  $D_1 = 22, d_2 = 27$
  - Release time:  $r_1 = 0, r_2 = 4$
  - Absolute Deadline:  $d_1 = 22, d_2 = 27$
- Start time:  $s_1 = 0, s_2 = 6$
- Finishing time:  $f_1 = 18, f_2 = 28$
- Lateness:  $L_1 = f_1 - d_1 = -4, L_2 = f_2 - d_2 = 1$
- Tardiness:  $E_1 = 0, E_2 = 1$

Each process  $P_i, i = 1, 2, \dots, 9$  is coded as follows-

```
repeat
  P(mutex)
  { Critical Section }
  V(mutex)
Forever
```

— wait —  
— signal —

The code for  $P_{10}$  is identical except that it uses  $V(mutex)$  in place of  $P(mutex)$ . What is the largest number of processes that can be inside the critical section at any moment?

Code for  $P_{10}$

```
repeat
  V(mutex)
  { Critical Section }
  V(mutex)
Forever
```

All 10 in CS

Suppose we want to synchronize two concurrent processes P and Q using binary semaphores S and T. The code for the processes P and Q is shown below-

Process P:

```
while(1) {
```

W:  $P(S)$

print '0';

print '0';

X:  $V(T)$

Process Q:

```
while(1) {
```

Y:  $P(T)$

print '1';

print '1';

Z:  $V(S)$

Make it short answer

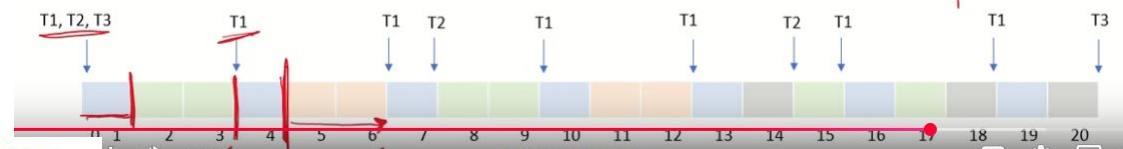
Synchronization statements can be inserted only at points W, X, Y and Z. Which of the following will always lead to an output string with '001100110011'?

- A.  $P(S)$  at W,  $V(S)$  at X,  $P(T)$  at Y,  $V(T)$  at Z, S and T initially 1
- B.  $P(S)$  at W,  $V(T)$  at X,  $P(T)$  at Y,  $V(S)$  at Z, S initially 1 and T initially 0
- C.  $P(S)$  at W,  $V(T)$  at X,  $P(T)$  at Y,  $V(S)$  at Z, S and T initially 1
- D.  $P(S)$  at W,  $V(S)$  at X,  $P(T)$  at Y,  $V(T)$  at Z, S initially 1 and T initially 0

इस हो वे केन सिंक्रोनाइज डी तू प्रोसेस  
तू रन वन आफ्टर दी अदर और

Consider a uniprocessor system executing three tasks (T1, T2 and T3, each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. Priority of each task is inverse of its period and available tasks are scheduled in order of priority, with highest priority task scheduled first. Each instance of T1, T2 and T3 requires an execution time of 1, 2 and 4 ms, respectively. Given that all tasks initially arrive at beginning of the 1st milliseconds and task preemptions are allowed, the first instance of T3 completes its execution at the end of \_\_\_\_\_ ms.

|    | Exe. Time | Period | Priority |
|----|-----------|--------|----------|
| T1 | 1         | 3      | P1       |
| T2 | 2         | 7      | P2       |
| T3 | 4         | 20     | P3       |



Suppose we want to synchronize two concurrent processes P and Q using binary semaphores S1 and S2. The code for the processes P and Q is shown below-

Process P:

```
while(1) {
```

$P(S1)$

$P(S2)$

Critical section

$V(S1)$

$V(S2)$

Process Q:

```
while(1) {
```

$P(S2)$

$P(S1)$

Critical section

$V(S1)$

$V(S2)$

S1

X 0

S2

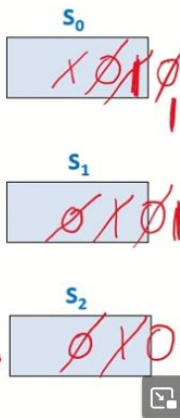
X 0

• This ensures-

- A. Mutual Exclusion
- B. Deadlock
- C. Both (A) and (B)
- D. None of these

The semaphores are initialized as  $S_0 = 1, S_1 = 0, S_2 = 0$ .

| Process $P_0$        | Process $P_1$      | Process $P_2$      |
|----------------------|--------------------|--------------------|
| while (true) {       | wait ( $S_1$ );    | wait ( $S_2$ );    |
| wait ( $S_0$ );      | release ( $S_0$ ); | release ( $S_0$ ); |
| print '0';           |                    |                    |
| release ( $S_1$ );   |                    |                    |
| release ( $S_2$ ); } |                    |                    |



A shared variable  $x$ , initialized to zero, is operated on by four concurrent processes  $W, X, Y, Z$  as follows. Each of the processes  $W$  and  $X$  reads  $x$  from memory, increments by one, stores it to memory and then terminates. Each of the processes  $Y$  and  $Z$  reads  $x$  from memory, decrements by two, stores it to memory, and then terminates. Each process before reading  $x$  invokes the P operation (i.e. wait) on a counting semaphore  $S$  and invokes the V operation (i.e. signal) on the semaphore  $S$  after storing  $x$  to memory. Semaphore  $S$  is initialized to two. What is the maximum possible value of  $x$  after all processes complete execution?

Execution:  $00$

A shared variable  $x$ , initialized to zero, is operated on by four concurrent processes  $W, X, Y, Z$  as follows. Each of the processes  $W$  and  $X$  reads  $x$  from memory, increments by one, stores it to memory and then terminates. Each of the processes  $Y$  and  $Z$  reads  $x$  from memory, decrements by two, stores it to memory, and then terminates. Each process before reading  $x$  invokes the P operation (i.e. wait) on a counting semaphore  $S$  and invokes the V operation (i.e. signal) on the semaphore  $S$  after storing  $x$  to memory. Semaphore  $S$  is initialized to two. What is the maximum possible value of  $x$  after all processes complete execution?

| Process W   | Process X | Process Y   | Process Z   |
|-------------|-----------|-------------|-------------|
| wait (S)    | wait (S)  | wait (S)    | wait (S)    |
| read (x)    | read (x)  | read (x)    | read (x)    |
| $x = x + 1$ |           | $x = x - 2$ | $x = x - 2$ |
| write (x)   |           | write (x)   | write (x)   |
| signal (S)  |           | signal (S)  | signal (S)  |

$S = 2$

| Process W   | Process X   | Process Y   | Process Z   |
|-------------|-------------|-------------|-------------|
| wait (S)    | wait (S)    | wait (S)    | wait (S)    |
| read (x)    | read (x)    | read (x)    | read (x)    |
| $x = x + 1$ | $x = x + 1$ | $x = x - 2$ | $x = x - 2$ |
| write (x)   | write (x)   | write (x)   | write (x)   |
| signal (S)  | signal (S)  | signal (S)  | signal (S)  |

Init  $S$  to 1, then mutex is achieved. Init  $S$  to 2, no mut

Consider the two functions incr and decr shown below.

incr() {

wait(s);  
X = X+1;  
signal(s); }

decr(){

wait(s);  
X = X-1;  
signal(s); }

$S = S - 1$

$S = S + 1$

There are 5 threads each invoking incr once, and 3 threads each invoking decr once, on the same shared variable X. Initial value of X is 10. Suppose there are two implementations of the semaphores, as follows:

I-1: s is a binary semaphore initialized to 1.

I-2: s is a counting semaphore initialized to 2.

Let V1, V2 be the values of X at the end of execution of all the threads with I-1, I-2, respectively. Which one of the following choices is a minimum possible values of V1, V2, respectively?

b) 7, 7

c) 12, 7

d) 12, 8

Let m[0]...m[4] be mutexes and P[0] .... P[4] be processes. Suppose each process executes the following:

```
wait (m[i]);
wait(m[(i+1) mod 4]);
-----
release (m[i]);
release (m[(i+1) mod 4]);
```

This could cause

(a) Thrashing ← X

(b) Deadlock

(c) Starvation

Suppose n processes, P<sub>1</sub>, ..., P<sub>n</sub> share (m) identical resource units, which can be reserved and released one at a time. The maximum resource requirement of process P<sub>i</sub> is s<sub>i</sub>, where s<sub>i</sub> > 0. Which one of the following is a sufficient condition for ensuring that deadlock does not occur?

(A)  $\forall i, s_i < m$

(B)  $\forall i, s_i < n$

(C)  $\sum s_i < (m + n)$  for  $i = 1$  to  $n$

(D)  $\sum s_i < (m * n)$  for  $i = 1$  to  $n$

$$\left. \begin{array}{l} P_1 \rightarrow S_1 - 1 \\ P_2 \rightarrow S_2 - 1 \\ \vdots \\ P_n \rightarrow S_n - 1 \end{array} \right\} < m$$

5 lawyers,  $S_i=2$ , at least 6 chopsticks

- In worst case, all processes acquire (s<sub>i</sub>-1) resources and need 1 more resource
- So following condition must be true to ensure that deadlock never occurs

$$\rightarrow \sum (s_i - 1) < m \quad \text{for } i = 1 \text{ to } n$$

$$\sum s_i < m + n \quad \text{for } i = 1 \text{ to } n$$

$$\sum_{i=1}^n (s_i - 1) < m$$

$$\sum_{i=1}^n s_i - n < m$$

# Quiz V Answer

- A. Yes – if the block size gets too big, fetches become more expensive and the big blocks force out more useful data.
- B. Certainly! That's call "tuning"
- C. "Most Recent" items  $\Rightarrow$  Temporal locality

- A. For a given cache size: a larger block size can cause a lower hit rate than a smaller one.
- B. If you know your computer's cache size, you can often **make your code run faster**.
- C. Memory hierarchies take advantage of **spatial locality** by keeping the most recent data items **closer** to the processor.

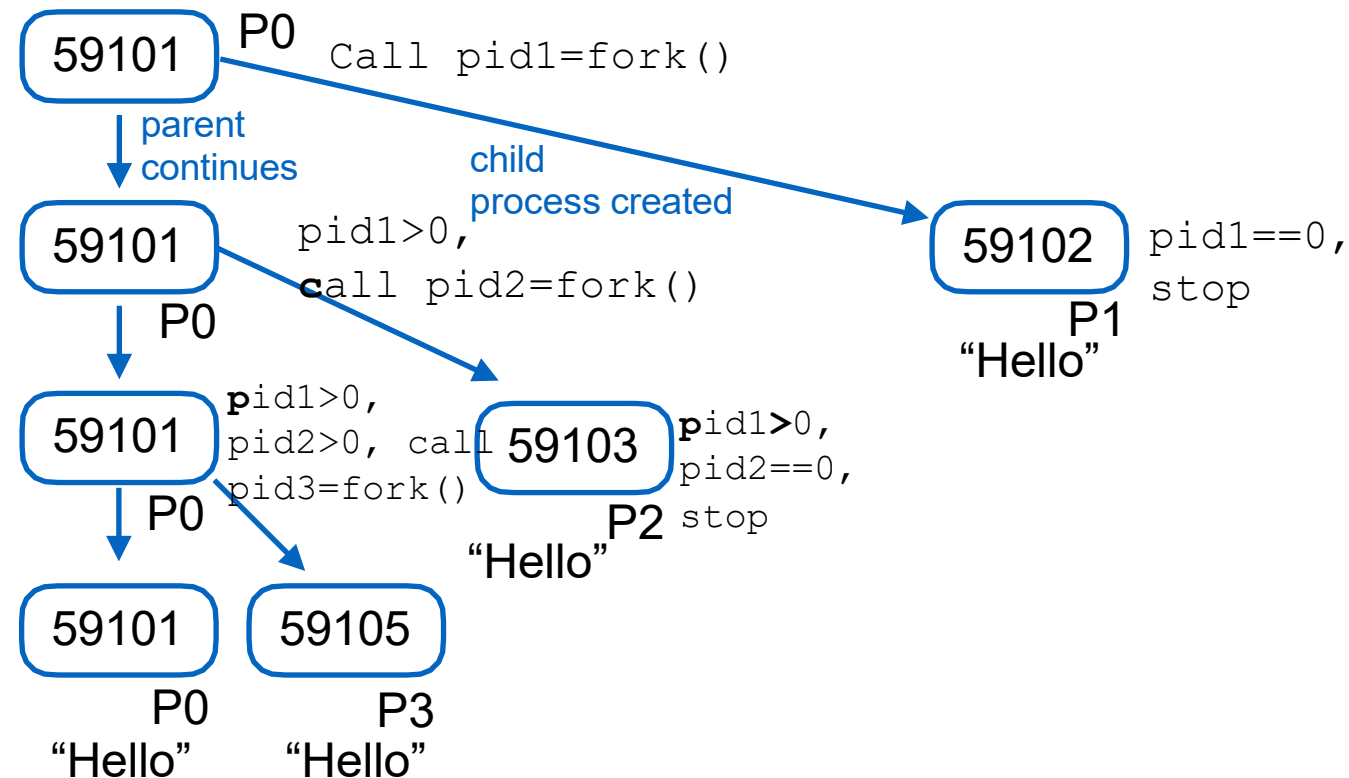
|     | ABC |
|-----|-----|
| 1 : | FFF |
| 1 : | FFT |
| 2 : | FTF |
| 2 : | FTT |
| 3 : | TFF |
| 3 : | TFT |
| 4 : | TTF |
| 5 : | TTT |

# Quiz con't

- For a given cache size, a larger block size can cause a lower hit rate than a smaller one.
  - True. A large block size leads to fewer cache blocks.
- If you know your computer's cache size, you can often make your code run faster.
  - True. By tuning your code to be cache-aware.
- Memory hierarchies take advantage of spatial locality by keeping the most recent data items closer to the processor.
  - False. This is called temporal locality.

# Quiz: Fc

```
1 #include <unistd.h>
2 int main (void) {
3     if( (pid1=fork()) >0 && (pid2=fork())
4         >0)
5         {pid3=fork();}
```



- Q: How many Hellos are printed out in total? Draw the process family tree. In C, the logical AND operator (&&) employs short-circuit evaluation, meaning it evaluates expressions from left to right and stops as soon as the result of the entire expression is determined. Specifically for (cond1&&cond2): If cond1 evaluates to false (zero), the overall result of the && operation is already known to be false, so cond2 is not evaluated. If cond1 evaluates to true (non-zero), the evaluation proceeds to the next operand cond2.
- A: There are 4 processes in total.
- The initial process P0 calls pid1=fork() to create child process P1. **P0 has** pid1>0, so it calls pid2=fork() to create child process P2.
- In **P1**, the if condition (pid1>0&&?) = (false&&?)=false, so p1 skips the call pid2=fork(), stops here and does not call any more fork(). (? stands for either true or false).
- The if condition (pid1>0&&pid2>0) is checked in both P0 and P2. It is (true&&true)=true in P0, so P0 calls pid3=fork() to create child process P3. It is (true&&false)=false in P2, so P2 stops here and does not call any more fork().

# Quiz: Fork

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    int x = 1;
    pid_t pid = fork();
    if (pid == 0){
        printf("Child has x = %d\n",
++x); }
    else{
        printf("Parent has x = %d\n",
--x); }
    return 0;
}
```

- Updates to variable x in the parent process and child process do not affect each other because they have separate address spaces, and separate copies of data variables.
- Since parent process and child process run concurrently without wait(), two output interleavings are possible.

Output:

Parent has x = 0

Child has x = 2

Or,

Child has x = 2

Parent has x = 0